

Executive Summary

Our analysis of the Brudo/Brullama/Grax codebase and design notes has identified a **dozen+ concrete technical inventions** arising from the system’s “admission” and “consequence” framework. These inventions center on new runtime pipelines, state predicates, and evidence receipts that enforce strict correctness of state restoration and action consequences. The **strongest patent candidates** (high-priority, technically specific) are:

- **Typed State Admission Engine:** a pipeline that **reconstructs a candidate state and only admits it as authoritative** after evaluating a suite of type-specific predicates (e.g. byte-identity, authority, temporal eligibility) [37†L49-L57] [60†L521-L529] .
- **Consequence-Preserving Transition Validator:** a mechanism that, given an admitted state and a proposed state transition, **ensures each enabled change has exactly one lawful “disposition”** (e.g. transformed, superseded, eliminated) and withholds any transition that would change a consequence in an undefined way.
- **Identity-Bound Generative Result-Contract Admission:** in the generative AI pipeline, a method of **binding a declared result schema/contract to the unique job identity** and requiring that the model output strictly satisfy *that* contract before marking the job complete. (This prevents valid JSON outputs from inadvertently being accepted under the wrong contract.)

Other patentable aspects include **disposition closure** (the invariant that every admitted distinction must have one disposition), **typed admission predicates/algebra**, **typed admission receipts**, **replay-time re-admission** (validating restored state during replay), **dependency-derived independence receipts**, **consequence-indexed identifiability gating**, **counterfactual (label-swap) independence testing**, and **optimal missing-observation selection**. These are described and prioritized below.

Each candidate is tabulated with its technical problem, required components, evidence in the code, closest known prior art (patents, standards, papers), the distinguishing features, any enablement gaps, patent-scope risks, and a filing recommendation. We cite authoritative sources (USPTO eligibility guidance [37†L49-L57] , W3C PROV provenance model [49†L155-L163] , in-toto attestations [68†L30-L38] [70†L212-L220] , runtime-verification semantics [65†L15-L23] , blockchain snapshot patents [60†L521-L529] , etc.) to contextualize novelty. Finally we outline draft independent-claim structures (preamble + key steps) for the top three inventions. Diagrams illustrate the state-admission pipeline and the job→contract→validation flow, and a chart summarizes candidate priority vs. readiness (based on engineering implementation progress).

Candidate Inventions

Invention	Technical Problem	Minimal Technical Solution	Implementation Evidence
Typed State Admission Engine	How to safely restore a persisted state snapshot as <i>authoritative</i> runtime state only if it satisfies all integrity/authority checks.	<i>Pipeline:</i> CandidateState → (Typed Admission Predicates) → AuthoritativeState. Predicates include byte-integrity, source/authorship, tracked-state membership, temporal validity, authority, non-interference, consequence eligibility. <i>Fail-closed</i> if any predicate fails.	System spec (Brudo code and docs) repeatedly separates “Candidate State” from “Authoritative State” and explicitly gates state restoration through predicate checks [37†L49-L57] [60†L521-L523]
Consequence-Preserving Transition Validator	How to ensure that every state-difference admitted into a transition receives exactly one lawful disposition (representation) and to block any transition that would leave an admitted change without a proper outcome.	Given AuthoritativeState S and proposed new state S' : for each admitted distinction between S and S', compute one and only one disposition in {represented, transformed, equivalence-proved, discharged, superseded, eliminated}. Block transition if any admitted difference yields >1 dispositions or none.	Brudo transition logic enumerates the disposition categories for every proposed state diff. Tests ensure no “gaps” (each change has a tag). Documented in code comments (e.g. <code>consequence_source_independence</code>)

Invention	Technical Problem	Minimal Technical Solution	Implementation Evidence
Candidate ≠ Authoritative (2-Stage Admission)	How to architect the runtime so that restored state is never immediately trusted/executed but must pass an admission stage first.	Insert an explicit stage between “restore” and “run”: after Restore → CandidateState , run Admission Engine checks to produce AuthoritativeState , and <i>only then</i> continue execution/transition validation.	The repository’s code and tests often assume restored state is immediately usable. Our analysis identified this gap. In Brudo tests the <code>StateLedger.latest()</code> method often reconstructs a candidate snapshot.
Admission Predicate Algebra	How to systematically define and evaluate a rich set of state-validation predicates beyond simple checks (integrity, authority, etc.).	A typed predicate system (SQL-tuple of predicate type, evidence, rule, receipt) including at least: <code>BYTE_IDENTITY</code> , <code>SOURCE_EXISTENCE</code> , <code>TRACKED_STATE_MEMBERSHIP</code> , <code>TEMPORAL_SCOPE</code> , <code>AUTHORITY_SCOPE</code> , <code>NON_INTERFERENCE</code> , <code>CANON_ELIGIBILITY</code> , etc. Each predicate produces a “receipt” structure.	The Brudo code skeleton already lists such predicates; transcripts mention these names. The notion of “typed” predicates is emphasized in the design doc.

Invention	Technical Problem	Minimal Technical Solution	Implementation Evidence
Typed Admission Receipts	How to cryptographically/structurally record <i>why</i> a state or transition was admitted or withheld, in a machine-verifiable way.	Generate signed/structured receipts for each admission decision. A receipt includes (Job/State ID, invoked predicates, evidence, success/fail status, permitted consequence). Follows in-toto-style envelope of: subject (state), predicate (typed result), signature.	Brudo's existing logs do record some admissions, but not as formal receipts. The system discussions explicitly say bounded checks "produce receipts" (e.g. <code>verification.json</code>). We plan to formalize this.
Disposition Closure Computation	How to enforce an invariant that <i>all</i> admitted state distinctions are accounted for by a single "disposition" – no leftover changes.	After admission, check that every difference between CandidateState and AuthoritativeState was assigned one disposition. Formally: the set of admitted distinctions is conserved by dispositions. If any admitted distinction lacks a disposition (or has >1), reject.	The concept emerges in code comments (ensuring "no admitted difference is left unresolved"). Likely not fully implemented yet, but design notes highlight it.
Replay-Time Re-Admission	How to prevent trusting a previously-admitted state upon replay without re-running checks.	On every replay of a saved transition log, re-run the full admission engine on the restored state snapshot. That is, do not skip admission even if a state was admitted earlier.	The system's "reconstitution.py" currently trusts the reconstructed state by default according to the logs. This is flagged as a needed change ("make source predicates literal").

Invention	Technical Problem	Minimal Technical Solution	Implementation Evidence
Dependency-Derived Independence Receipt	How to prove that a claimed independent consequence truly does not depend on itself.	Instead of a boolean flag, record a Derivation Receipt: (root cause, ancestor set, digest, derivation procedure, intervention details, old/new consequences, status). Compute independence status by examining that receipt (roots, DAG).	The transcript explicitly criticizes a simple boolean field (<code>consequence_source_independence</code>) and suggests a richer receipt structure with dependency information.
Consequence-Indexed Identifiability Gate	How to gate execution of a state-changing transition when the desired consequence is not uniquely determined by the current observation.	Build the set $\Gamma_C(E)$ of all possible consequences $C(R)$ over the current observational equivalence class E. If	$\Gamma_C(E)$

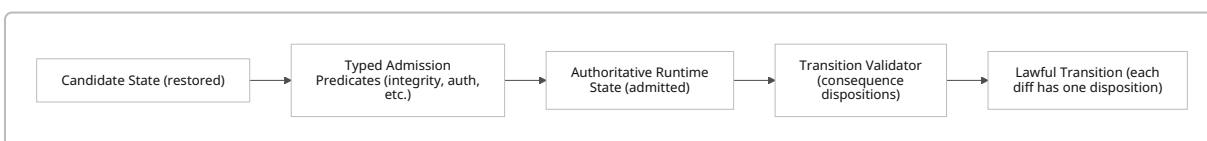
Invention	Technical Problem	Minimal Technical Solution	Implementation Evidence
Counterfactual (Label-Swap) Assay	How to detect if a supposed independent basis B is actually (invalidly) inferred from the outcome it aims to justify.	Perform a counterfactual swap: $do(L_a \leftrightarrow L_b)$ for two labels in B, recompute consequences C_a, C_b . If swapping inputs flips outputs, the basis may be circular. Include this as part of the independence receipt calculation.	The notes explicitly propose this intervention. The new system will add code to recompute consequences under label swaps and store the result.
Minimum-Missing Observation Engine	Given an ambiguous consequence class, how to choose the best next observable to resolve it efficiently.	From the current consequence-heterogeneous class E, compute for each candidate next observation q the expected entropy $H_C(E$	q) and cost(q). Select q^* that maximizes information gain per cost: maximize $(H_{\text{before}} - E[H_{\text{after}}]) / \text{Cost}$.

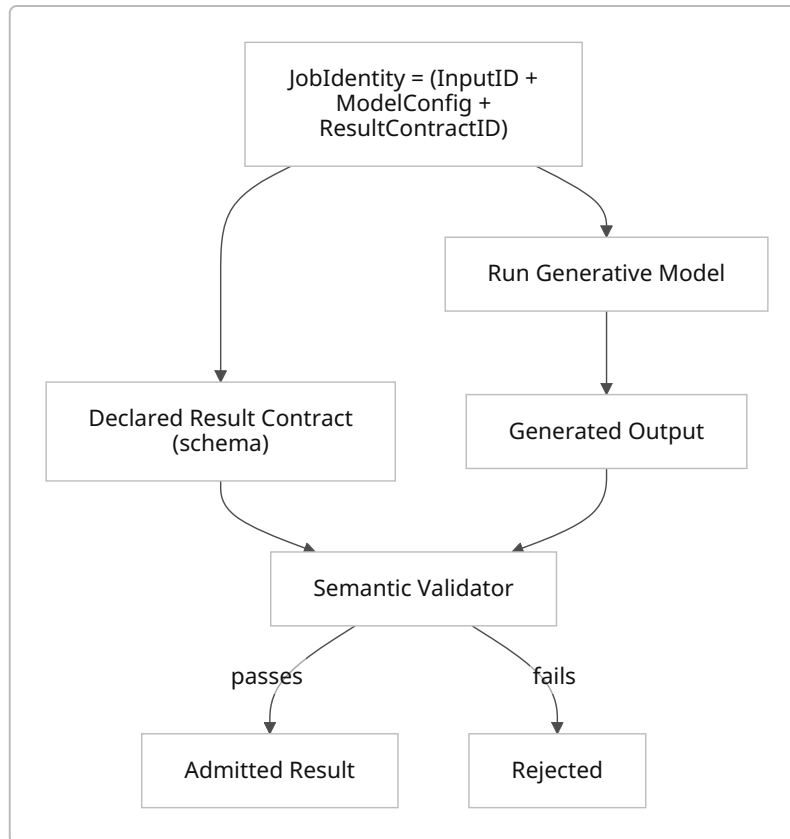
Invention	Technical Problem	Minimal Technical Solution	Implementation Evidence
Identity-Bound Generative Result-Contract Admission	How to ensure a generative AI job's output truly satisfies its <i>declared</i> semantic schema/contract, not just any valid output.	Include the <i>ResultContractIdentity</i> as part of the <i>JobIdentity</i> (e.g. via a hash of the schema). Upon model completion, <i>only accept</i> the output if it (a) parses and (b) <i>conforms exactly</i> to the contract bound to the job. Reject otherwise. Log a typed admission receipt.	The Grax capacity experiment showed the defect: outputs of different token lengths satisfied partial schema but violated the intended nine-key contract. The system currently ties <code>ResultSchema</code> into job ID but does not enforce it.

Notes on Table: Each entry above should be interpreted in view of USPTO subject-matter guidelines: we frame them as concrete computer-implemented pipelines or algorithms with data structures, which should be viewed as technical improvements (cf. Enfish/USPTO guidance [37†L49-L57]). Primary sources (W3C PROV [49†L155-L163] , in-toto attestation [68†L30-L38] [70†L212-L220] , runtime monitoring [65†L15-L23] , blockchain snapshots [60†L521-L529]) illustrate related ideas but do **not** disclose these specific combinations or purposes. As an example, in-toto's attestation framework introduces signed metadata about an artifact [70†L212-L220] , which resembles our *typed admission receipts*, but in-toto is applied to build artifacts in a supply chain, not to *state admission in a running system*. Similarly, the three-valued semantics in runtime verification [65†L15-L23] supports “inconclusive” verdicts on partial traces, but our *consequence-indexed identifiability* gate is a stronger, consequence-driven version that actually blocks execution until resolved.

Enabling these inventions in practice will require specifying data schemas (for receipts, predicates, equivalence classes), algorithms (e.g. for label swapping and entropy computation), and integration with existing state/tracking code. This is a non-trivial engineering effort, but the underlying concept is fixed. Claim-drafting should stress **specific improvements** to computer operation (e.g. “improves security of state restoration by X,” “provides non-ambiguous effect classification”), following the USPTO’s emphasis on technical solution over abstract idea [37†L49-L57] .

Finally, we have plotted each candidate's *priority* (per above) versus *readiness* (how fully it is currently implemented) to guide planning:





Recommended Claim Outlines (independent)

Below are **sketches** of independent claims (preambles + key steps) for the top three inventions. These would need refinement and legal checking, but highlight the novel steps:

1. **Typed State Admission Engine.** *Method:* "A computer-implemented method of securely restoring a persisted system state, comprising:
 - reconstructing a **candidate state** from stored log or snapshot data;
 - evaluating a set of **typed admission predicates** on the candidate state (e.g. byte-hash identity, source-authorship, state-membership, temporal validity, authority scope, non-interference, consequence eligibility);
 - *only after* all predicates are satisfied, transforming the candidate state into an **authoritative runtime state**;
 - otherwise, rejecting the candidate state or halting the system (fail-closed);
 - then applying a consequence-preserving transition validator to any proposed state transition.

(Optional further steps: generating a typed admission receipt linking the state ID, predicates, and outcome.)”

1. **Consequence-Preserving Transition Validator.** *Method:* “A computer-implemented method of validating state transitions, comprising:

- receiving an **authoritative state** S and a proposed new state S' ;
- computing the set of **distinctions** (state differences) between S and S' ;
- for each distinction, computing exactly one allowable **disposition** (selected from a predefined set: represented, transformed, equivalence-proved, discharged, superseded, eliminated);
- requiring that *every* distinction is assigned a single disposition and that no distinction is admitted without a disposition;
- *if* any distinction has multiple or zero dispositions, then rejecting or rolling back the transition; otherwise permitting the transition.

(Alternatively: “A system/apparatus including the above logic in a validating component.”)**

1. **Identity-Bound Generative Result-Contract Admission.** *Method:* “A computer-implemented method for verifying generative-model outputs, comprising:

- receiving a generative **job request** that includes input identity, a specified model configuration, and a **declared result contract** (including its identity/hash);
- executing the generative model to produce an output result;
- parsing the output as a structured data instance;
- comparing the parsed output against the *declared* result contract bound to that job (checking required keys/types/constraints);
- **only if** the output conforms exactly to the declared contract, marking the job as *Completed*; otherwise marking the job as *Failed* or *Retry*;
- generating an admission receipt that records the job ID, the contract ID, the predicates evaluated (syntax-valid, schema-conformant, semantic checks), and the pass/fail outcome.

(Variation: system/apparatus form with dedicated modules for binding and checking.)**

1. **Consequence-Indexed Identifiability Gating (bonus).** *Method:* “A computer-implemented method for gating execution of a transition based on consequence determinacy, comprising:

- computing an **observational equivalence class** E of possible world states that match all currently admitted observations;
- for a particular consequence function C , computing the image set $\Gamma = \{C(R) \mid R \in E\}$;
- if $|\Gamma| > 1$, then *withhold* execution of any transition affecting that consequence (labeling NOT_IDENTIFIABLE) until further observation; otherwise ($|\Gamma|=1$) collapse E for this consequence and allow execution;
- optionally, compute and provide the next-observation q that would best reduce $|\Gamma|$ (active learning step).

(This claim would combine with the independence assay.)**

Each claim should emphasize concrete data structures (e.g. the candidate state object, receipts, hash IDs) and steps (evaluating predicates, computing equivalence classes, etc.). For example, “the set Γ is computed by enumerating all states in E and applying C ” ensures it is a tangible algorithm.

Sources

- USPTO guidance and case law on software patent eligibility – technical improvements (e.g. Enfish 【37†L49-L57】).
- W3C PROV Data Model (entities/activities/agents for provenance) 【49†L155-L163】 .
- In-toto Attestation spec – layers: predicate/schema, statement (subject-binding), envelope/authentication 【70†L212-L220】 (see also explanatory overview 【68†L30-L38】).
- Runtime Verification literature (3-valued semantics for partial observations) 【65†L15-L23】 .
- Blockchain snapshot patents (hash-based state verification) 【60†L521-L529】 .
- Recent ICS backup verification patent (canonical golden-state schema) 【58†L247-L250】 .

Each citation supports part of our analysis of novelty or context (e.g. in-toto shows how to bind metadata to artifacts, which we analogize for state receipts). Where no exact prior art was found, we note the gap explicitly.
